



51. Machine Learning Workflow

Your guide to the most comprehensive Machine Learning workflow with Python

Previous Section:

 [50. Reinforcement Learning](#)

Contents:

[1. Problem Statement](#)

[2. Data Preparation: Missing Values](#)

[Step 1: Identify Numerical and Categorical Columns](#)

[Step 2: Binary vs Multi-Class](#)

[Step 3: The Hidden Problem with Integers](#)

[Step 4: Detect Nullable Integer Columns](#)

[Step 5: Remaining Numerical Columns](#)

[Step 6: Apply Imputation](#)

[Step 7: Verification](#)

[3. Data Preparation: Sampling](#)

[Step 1: Separate Classes](#)

[Step 2: Oversample Minority Class](#)

[Step 3: Combine Back](#)

[Step 4: Verify Distribution](#)

[4. Data Preparation: Feature Scaling & Encoding](#)

[Step 1: Encoding Categorical Features](#)

[Step 2: Scaling Numerical Features](#)

[5. Supervised Learning](#)

[Step 1: Train-Test Split](#)

[Step 2: Define Evaluation Function](#)

[Step 3: Train Multiple Models](#)

[Step 4: Final Results](#)

[6. Unsupervised Learning](#)

[Clustering with K-Means](#)

[7. Semi-Supervised Learning: Label Propagation](#)

[Label Propagation](#)

[Visualization](#)

[Summary: Three Ways of Learning](#)

[References](#)

Machine Learning is not just about building models—it is about understanding data, observing patterns, and turning raw information into decisions that actually work in the real world. In this guide, we are not treating algorithms as isolated tools, but as different ways of learning from the same problem. The goal is simple: to explore how machines learn under different assumptions, and to see what happens when we let them approach the same dataset from multiple perspectives.

To do that, we will work through three major learning paradigms. Each one represents a different philosophy of learning:

- **Supervised Learning** focuses on learning from labeled data, making it suitable for classification and regression tasks.
- **Unsupervised Learning** works without labels, discovering hidden structures such as clusters within the data.
- **Semi-Supervised Learning** bridges the gap, combining a small amount of labeled data with a larger pool of unlabeled data to improve predictions.

Rather than studying these approaches separately, we will train, evaluate, and compare them on the same dataset. This allows us to move beyond theory and actually see their strengths, limitations, and behavior in practice. Some models will perform better under structure, others will adapt better to ambiguity, and some will reveal patterns that are not immediately visible.

This workflow is designed as a journey. We begin with a problem, move through data preparation and modeling, and end with evaluation and comparison. By the end, the objective is not just to build models, but to understand **why** they behave the way they do—and how to choose the right approach when facing a real-world problem.

1. Problem Statement

Before we even think about models, we observe. That is where everything begins.

I was given a dataset:

[college_placement.csv](#)

This is something that looks clean at first glance, but like most real-world data, it hides imperfections beneath the surface. It is a synthetic dataset of over 10,000 students, designed

to simulate college placement outcomes. Each row represents a student. Each column captures a part of their story—academic results, skills, experience, and potential.

We are not just looking at numbers. We are trying to understand **what influences whether a student gets placed or not**.

To begin, we load the dataset and inspect it carefully.

```
import pandas as pd

def show_info(df, target, features):
    # Get rows and columns
    print(f"Dataset Basic Info:")
    print("-" * 50)
    print(f"+ Rows: {df.shape[0]}; Columns: {df.shape[1]}")
    print(f"+ Unique Labels: {df[target].nunique()}")

    label_counts = {label: count for label, count in df[target].value_counts().items()}
    print(f"+ Value counts of each label\n: {label_counts}")
    print(f"+ Features: {features}")
    print("-" * 50)

    # Show info
    df.info()

# Read the dataset
df = pd.read_csv('college_placement.csv')
# Inspect dataset
show_info(df, target='Placement', features=df.columns.drop('Placement'))
```

And this is what we observe when running this program:

- The dataset contains 10,200 rows and 9 columns
- The target variable is Placement, with 2 unique labels: Yes and No
- The distribution is highly uneven: No: 8457; Yes: 1743

The features describe multiple aspects of a student:

- Academic signals: CGPA, previous semester result, academic performance
- Experience: internships, projects completed
- Personal traits: communication skills, extracurricular score
- Context: major

But observation does not stop at structure. I look deeper—at what is missing. From `df.info()`, several columns have fewer non-null values than expected. That means one thing: missing

values exist. And now, the dataset reveals two immediate problems:

- Missing values across multiple features
- A clear imbalance in the target labels

Now comes the decision. Which problem do we solve first? I choose to handle missing values first. Because incomplete data breaks everything that comes after. Sampling, training, even simple statistics—they all assume the data exists. If values are missing, any attempt to rebalance or model the dataset becomes unreliable.

So the workflow becomes clear: We do not rush into modeling. We fix the foundation first.

2. Data Preparation: Missing Values

Now that the problem is clear, I begin organizing the data. Before fixing anything, I need to understand what each column represents—because not all missing values should be treated the same way.

So first, I separate **target** and **features**.

```
# Define target and features
target = 'Placement'
features = df.columns.drop(target)

X = df[features]
y = df[target]
```

From here, I start classifying the columns. Not by name, but by their nature.

Step 1: Identify Numerical and Categorical Columns

```
# Numerical and categorical columns
num_cols = df.select_dtypes(include=['float64', 'int64']).columns.tolist()
cat_cols = df.select_dtypes(include=['object']).columns.tolist()

print("Numerical Columns:", num_cols)
print("Categorical Columns:", cat_cols)
```

What I observe:

- **Numerical Columns** ['IQ', 'Prev_Sem_Result', 'CGPA', 'Academic_Performance', 'Extra_Curricular_Score', 'Communication_Skills', 'Projects_Completed']
- **Categorical Columns** ['Major', 'Internship_Experience', 'Placement']

But this is still not enough. Some categorical columns behave very differently from others.

Step 2: Binary vs Multi-Class

```
# Identify binary and multi-class categorical columns
binary_cols = [col for col in cat_cols if df[col].nunique() == 2 and col != target]
multi_class_cols = [col for col in cat_cols if df[col].nunique() > 2]

print("Binary Columns:", binary_cols)
print("Multi-Class Columns:", multi_class_cols)
```

Output:

```
Binary Columns: ['Internship_Experience']
Multi-Class Columns: ['Major']
```

Observation:

- **Binary Columns:** ['Internship_Experience']
- **Multi-Class Columns:** ['Major']

This distinction matters. Binary columns are simple—they only have two states. Missing values here should not be guessed with averages, they should follow the most frequent pattern.

So I make a decision:

- Binary → **Mode imputation**
- Multi-class → handled later (encoding stage)

Step 3: The Hidden Problem with Integers

Now comes something subtle, but important. When integers contain missing values, pandas silently converts them into floats. For example:

```
[1, 2, 3, NaN] → becomes float64
```

That might look harmless. But it breaks meaning.

If a column represents counts, scores, or discrete values... introducing decimals (via mean or median imputation) corrupts the data. A student cannot complete **3.7 projects**. That number does not exist.

So instead of treating them as floats, I test whether they were originally integers.

Step 4: Detect Nullable Integer Columns

```
# Detect nullable integer columns
nullable_int_cols = []

for col in num_cols:
    try:
        df[col].astype('Int64') # Try converting
        nullable_int_cols.append(col)
    except:
        pass

print("Nullable Integer Columns:", nullable_int_cols)
```

What I find:

- **Nullable Integers:** ['IQ', 'Academic_Performance', 'Extra_Curricular_Score', 'Communication_Skills', 'Projects_Completed']

These are not continuous values. They are discrete. So I treat them carefully.

Decision: Nullable integers → **Mode imputation (preserve integer nature)**

Step 5: Remaining Numerical Columns

After removing nullable integers, what remains are true continuous variables.

```
# Remaining numerical columns
remaining_num_cols = [col for col in num_cols if col not in nullable_int_cols]

print("Remaining Columns:", remaining_num_cols)
```

Result:

- **Remaining Columns:** ['CGPA', 'Prev_Sem_Result']

These are safe for mean imputation, because decimal values make sense here

Step 6: Apply Imputation

Now I apply everything systematically.

```
# Impute binary and multi columns with mode
for col in binary_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

for col in multi_class_cols:
```

```

df[col] = df[col].fillna(df[col].mode()[0])

# Impute nullable integer columns with mode
for col in nullable_int_cols:
    df[col] = df[col].astype('Int64')
    df[col] = df[col].fillna(df[col].mode()[0])

# Impute remaining numerical columns with mean
for col in remaining_num_cols:
    df[col] = df[col].fillna(df[col].mean())

```

Step 7: Verification

I do not assume it worked. I verify.

```

# Check missing values again
print(df.isnull().sum())

# Check data types
df.info()

```

Now the dataset tells a different story:

- No missing values remain
- Integer columns are restored to proper types
- Continuous columns retain their numeric meaning

Throughout the process, I did not just “fill in the blanks”. I preserved the **structure of the data**. Because in Machine Learning, small mistakes at this stage do not stay small. They propagate. They amplify. They distort everything that follows. So I fix the data, before the model ever sees it.

3. Data Preparation: Sampling

Now that the data is complete, I move to the second problem I observed earlier.

The dataset is not balanced. Most students are labeled **“No”**, while only a small portion are labeled **“Yes”**. This creates a bias. If I train a model like this, it can achieve high accuracy by simply predicting **“No”** most of the time. That is not intelligence. That is avoidance.

So I correct the distribution. Not by removing data, but by **giving more presence to what is rare**. I choose **oversampling**. Instead of discarding information from the majority class, I replicate samples from the minority class until both sides are comparable. It is simple and controlled. And in this case, effective.

Step 1: Separate Classes

```
import numpy as np

# Combine features and target for easier manipulation
df_combined = df.copy()

# Separate majority and minority classes
majority_class = df_combined[df_combined[target] == 'No']
minority_class = df_combined[df_combined[target] == 'Yes']

print("Before Sampling:")
print(df_combined[target].value_counts())
```

Step 2: Oversample Minority Class

I replicate the minority samples randomly until they match the majority size.

```
# Oversample minority class
minority_oversampled = minority_class.sample(n=len(majority_class), replace=True, random_state=42)
```

Step 3: Combine Back

```
# Combine majority and oversampled minority
df_balanced = pd.concat([majority_class, minority_oversampled])

# Shuffle dataset
df_balanced = df_balanced.sample(frac=1, random_state=42).reset_index(drop=True)
```

Step 4: Verify Distribution

```
print("After Sampling:")
print(df_balanced['Placement'].value_counts())
```

Now, the dataset is balanced:

- The number of “**Yes**” and “**No**” samples are equal
- The model will no longer be biased toward the majority

I did not create new knowledge. I simply **allowed the model to see enough of what was previously rare**. Oversampling does not make the data “better.” It makes the learning process **fair**. And that is enough for the next step.

4. Data Preparation: Feature Scaling & Encoding

Before I move forward, there is one principle I always keep in mind. No matter how advanced a model sounds, no matter how complex the algorithm is, **it does not understand words, categories, or meaning**. It only understands **numbers**. A single number. A vector of numbers. A matrix. Even higher-dimensional tensors.

But in the end, everything must be translated into a numerical form. So now, I prepare the data in a way the model can actually interpret.

Step 1: Encoding Categorical Features

Before anything else, I translate categories into numbers. Because the model cannot interpret words—it only understands structure.

The column Major contains categorical values. These are labels, not quantities. If I assign numbers like 1, 2, 3..., I would accidentally introduce relationships that do not exist.

```
# One-Hot Encoding for 'Major'
# dtype=int makes values 0 and 1 instead of True/False)
df_encoded = pd.get_dummies(df_balanced, columns=['Major'], dtype=int)
```

After this step:

- The original Major column disappears
- New columns appear like:
 - Major_Computer_Science
 - Major_Electrical
 - Major_Mechanical
- Each row now carries its category in binary form

No ambiguity. No artificial ordering. Just pure representation.

Handling Binary Categorical Feature - Now I turn to `Internship_Experience`. This is different. It is not multi-class—it is **binary**. So One-Hot Encoding would be unnecessary. It would create redundant columns. Instead, I map it directly.

```
# Map binary column
df_encoded['Internship_Experience'] = df_encoded['Internship_Experience'].map({'Yes': 1, 'No': 0})
```

Now: **Yes** → **1**; **No** → **0**. Simple. Clean. No extra dimensions.

Target Encoding (Special Case)

Normally, I do not touch the target—but since it is categorical, I map it once:

```
df_encoded[target] = df_encoded[target].map({'Yes': 1, 'No': 0})
```

Not all categorical data should be treated the same way.

- Multi-class → expand into multiple dimensions
- Binary → compress into a single signal

Because encoding is not just conversion. It is **preserving meaning while changing form**.

Step 2: Scaling Numerical Features

Now I look at the numerical features. They are not on the same scale:

- CGPA numerically range from 0 to 10
- IQ might range from 0 to 100
- Projects completed might be small integers

If I leave them like this, some features will dominate others purely because of their scale. That is not learning. That is imbalance. So I standardize them.

Standardization transforms each feature so that: Mean = 0; Standard Deviation = 1. This ensures all features contribute **fairly**.

```
from sklearn.preprocessing import StandardScaler

# Separate features and target again
X = df_encoded.drop(target, axis=1)
y = df_encoded[target]

# Identify numerical columns (after encoding)
num_cols = X.select_dtypes(include=['int64', 'float64']).columns

# Apply Standardization
```

```
scaler = StandardScaler()
X[num_cols] = scaler.fit_transform(X[num_cols])
```

At this point, the data has been fully translated.

- Categorical values → converted into binary numerical form
- Numerical values → scaled into a consistent range

Everything is now in the same language. Because a model does not “see” students. It does not “understand” majors or internships. It only sees patterns in numbers. And now, the data is ready to speak that language.

5. Supervised Learning

Now the data is ready, so I finally allow the models to learn. This is **Supervised Learning**—I give the model examples with answers, and it learns the relationship between features and the target: **Placement**.

Step 1: Train-Test Split

I separate the data into two parts:

- **80% Training** → where the model learns
- **20% Testing** → where I evaluate its understanding

```
from sklearn.model_selection import train_test_split

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42, stratify=y)
```

Step 2: Define Evaluation Function

Instead of repeating myself, I build a function.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

def train_evaluate_model(model, name):
    print("Model:", name)

    # Train
    model.fit(X_train, y_train)
```

```

# Predict
y_pred = model.predict(X_test)

# Evaluate
conf_matrix = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1score = f1_score(y_test, y_pred)

print(f"Confusion Matrix:\n{conf_matrix}")
print(f"Accuracy: {accuracy}")
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F1-Score: {f1score}\n")

return conf_matrix, {"Model": name, "Accuracy": accuracy, "Precision": precision,
                    "Recall": recall, "F1-Score": f1score}

```

Step 3: Train Multiple Models

I do not rely on one perspective. I test four:

- Logistic Regression → linear thinking
- Decision Tree → rule-based decisions
- Random Forest → ensemble of trees
- Gradient Boosting → sequential refinement

```

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

models = {
    LogisticRegression(solver='liblinear', random_state=42): 'Logistic Regression',
    DecisionTreeClassifier(criterion='entropy', random_state=42): 'Decision Tree',
    RandomForestClassifier(n_estimators=100, random_state=42): "Random Forest",
    GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, max_depth=5, random_state=42): "Gradient Boosting"
}

```

```

results = []
cfm_results = []

for model, name in models.items():
    cfm, res = train_evaluate_model(model, name)
    results.append(res)
    cfm_results.append(cfm)

```

Step 4: Final Results

After observing all models, I summarize everything.

Model	Confusion Matrix	Accuracy	Precision	Recall	F1 Score
Logistic Regression	[[1421, 276], [206, 1480]]	0.8575	0.8428	0.8778	0.8600
Decision Tree	[[1687, 10], [1, 1685]]	0.9967	0.9941	0.9994	0.9967
Random Forest	[[1696, 1], [1, 1685]]	0.9994	0.9994	0.9994	0.9994
Gradient Boosting	[[1695, 2], [2, 1684]]	0.9988	0.9988	0.9988	0.9988

I gave four models the same data. But they did not learn the same way.

- **Logistic Regression:** It struggles. Not because it is weak... but because it is limited. It sees the world through a **linear boundary**, and this problem is clearly more complex.
- **Decision Tree:** It improves dramatically. It captures **non-linear relationships** and adapts to the structure of the data.
- **Random Forest & Gradient Boosting:** They dominate. Not by being smarter individually, but by **combining multiple weak learners into a strong one**.
 - Random Forest → reduces variance through averaging
 - Gradient Boosting → reduces error step by step

The difference is no longer small.

- From **85% → 99.9% accuracy**
- From simple assumptions → to layered, collective learning

So the conclusion becomes sharper: A model does not fail because it is "bad". It fails because **its way of seeing the data is too simple for the problem**.

6. Unsupervised Learning

Now, I stop giving answers to the model. I let it observe. This is **Unsupervised Learning**—no labels, no guidance. Just patterns hidden inside the data.

I choose **K-Means Clustering**. Its goal is simple: group similar data points together. Not to predict, but to **reveal structure**. For this dataset, I focus on two signals: **IQ and CGPA**. Because sometimes, intelligence and academic performance already tell a story.

Clustering with K-Means

```
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

def clustering_model(x, y, clusters=3):
    # Extract relevant features
    data = df[[x, y]]

    # Standardize
    scaler = StandardScaler()
    data_scaled = scaler.fit_transform(data)

    # K-Means
    kmeans = KMeans(n_clusters=clusters, random_state=42)
    kmeans.fit(data_scaled)

    # Assign clusters
    df['Cluster'] = kmeans.labels_

    # Evaluate clustering
    sil_score = silhouette_score(data_scaled, kmeans.labels_)
    print(f"Silhouette Score: {sil_score:.4f}")

    # Plot
    plt.figure(figsize=(8,6))
    plt.scatter(df[x], df[y], c=df['Cluster'], cmap='viridis', s=50)
    plt.xlabel(x)
    plt.ylabel(y)
    plt.title(f'K-Means Clustering: {x} vs {y}')
    plt.colorbar(label='Cluster')
    plt.show()

# Run clustering
clustering_model(x='IQ', y='CGPA', clusters=5)
```

I did not tell the model what “good” or “bad” means. Yet it still separated students into groups:

- High IQ + High CGPA
- Low IQ + Low CGPA
- Mixed patterns in between

This is the strength of unsupervised learning. It finds **structure without instruction**.

7. Semi-Supervised Learning: Label Propagation

Now I do something different. I give the model **partial truth**, and ask it to infer the rest. This is **Semi-Supervised Learning**—a balance between knowing and guessing.

Most real-world data is like this: A small portion is labeled. A large portion is not. So instead of wasting unlabeled data, I use it.

Label Propagation

This method treats data as a graph:

- Each sample → a node
- Similar samples → connected
- Labels → spread across the graph

```
from sklearn.semi_supervised import LabelPropagation
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score, adjusted_rand_score
from sklearn.preprocessing import StandardScaler

# Copy labels
y_unlabeled = df_encoded[target].copy()

# Remove 80% of labels
unlabeled_idx = df_encoded.sample(frac=0.8, random_state=42).index
y_unlabeled.loc[unlabeled_idx] = -1

# Features
X = df.drop(columns=[target]).values
y_semi_array = y_unlabeled.values

# Scale
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```

# Model
model = LabelPropagation(kernel='rbf', gamma=20, max_iter=5000)
model.fit(X_scaled, y_semi_array)

# Predictions
y_pred = model.transduction_

# Evaluation
y_true = df[target].values

print("Confusion Matrix:\n", confusion_matrix(y_true, y_pred))
print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall:", recall_score(y_true, y_pred))
print("F1 Score:", f1_score(y_true, y_pred))
print("ARI:", adjusted_rand_score(y_true, y_pred))

```

Visualization

```

# Mask labeled data
mask = y_semi_array != -1
y = df[target].values

import matplotlib.pyplot as plt
fig, ax = plt.subplots(1, 2, figsize=(12, 4))

# Before
ax[0].scatter(X_scaled[:, 0], X_scaled[:, 1], c='lightgray', s=30, label='Unlabeled')
ax[0].scatter(X_scaled[mask, 0], X_scaled[mask, 1], c=y[mask], cmap='viridis', s=60, label='Labeled')
ax[0].set_title("Before propagation – few labels")
ax[0].legend()

# After
ax[1].scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_pred, cmap='viridis', s=60)
ax[1].set_title("After propagation – all labeled")

plt.tight_layout()
plt.show()

```


At first, most data points are silent. After propagation, everything has a label. The model did not invent new knowledge. It extended what little it knew, across similarity.

(Below is the full code with necessary adjustments for readability and an error-free output. The code might run slowly depending on the environment):

```
machine\_learning\_workflow.py
```

Summary: Three Ways of Learning

I have now explored three different ways machines learn. Not just algorithms, but philosophies.

1. Supervised Learning — Learning with Answers

- Uses labeled data
- Best for: **Prediction tasks** (classification, regression)
- Strength: High accuracy when data is well-labeled
- Weakness: Requires large labeled datasets

2. Unsupervised Learning — Learning without Answers

- No labels at all
- Best for: **Pattern discovery, clustering, exploration**
- Strength: Reveals hidden structure
- Weakness: No guarantee clusters match real-world meaning

3. Semi-Supervised Learning — Learning with Partial Truth

- Mix of labeled and unlabeled data
- Best for: **Situations with limited labels**
- Strength: Uses all available data efficiently
- Weakness: Can propagate errors if initial labels are weak

I gave the same dataset three different perspectives: One learned from **answers**; One learned from **patterns**; One learned from **connections**

And each one revealed something different. That is the real lesson. Machine Learning is not about choosing the “best” model. It is about understanding **how the model learns**, and when that way of learning is appropriate.

So when you face a real problem, ask yourself:

- Do I have labels?
- Do I have structure?
- Or do I only have fragments of truth?

Because the answer to that question, determines everything. And this is where the workflow ends. Not as a conclusion, but as a foundation.

References

<https://www.datacamp.com/blog/a-beginner-s-guide-to-the-machine-learning-workflow>

<https://www.purestorage.com/knowledge/machine-learning-workflow.html>

Next Section:

 [52. Scikit-Learn Library](#)